JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.reflect

Class Constructor<T>

java.lang.Object
  java.lang.reflect.AccessibleObject
    java.lang.reflect.Executable
      java.lang.reflect.Constructor<T>

Type Parameters:

`T - the class in which the constructor is declared`

All Implemented Interfaces:

`AnnotatedElement`, `GenericDeclaration`, `Member`

```
public final class Constructor<T>
extends Executable
```

`Constructor` provides information about, and access to, a single constructor for a class.

`Constructor` permits widening conversions to occur when matching the actual parameters to newInstance() with the underlying constructor's formal parameters, but throws an `IllegalArgumentException` if a narrowing conversion would occur.

See Also:

`Member`, `Class`, `Class.getConstructors()`, `Class.getConstructor(Class[])`, `Class.getDeclaredConstructors()`

Field Summary

Fields inherited from interface java.lang.reflect.Member

`DECLARED`, `PUBLIC`

Method Summary

Concrete Methods

Modifier and Type	Method	Descriptio
boolean	**equals**(**Object** obj)	Compares Construc the specif
AnnotatedType	**getAnnotatedReceiverType**()	Returns a Annotate object tha represent a type to s receiver t method/co represent Executabl
AnnotatedType	**getAnnotatedReturnType**()	Returns a Annotate object tha represent a type to s return typ method/co represent Executabl
<T extends **Annotation**> T	**getAnnotation**(**Class**<T> annotationClass)	Returns th element's for the sp if such an is *present*
Annotation[]	**getDeclaredAnnotations**()	Returns a that are *d present* o element.
Class<**T**>	**getDeclaringClass**()	Returns th object rep the class that decla executabl represent object.
Class<?>[]	**getExceptionTypes**()	

		Returns a `Class` obj represent of excepti declared t thrown by underlyin executabl represent object.
`Type[]`	`getGenericExceptionTypes()`	Returns a Type obje represent exception to be thro executabl
`Type[]`	`getGenericParameterTypes()`	Returns a Type obje represent paramete declaratio the execu represent object.
`int`	`getModifiers()`	Returns th language for the ex represent object.
`String`	`getName()`	Returns th this const string.
`Annotation[][]`	`getParameterAnnotations()`	Returns a arrays of `Annotati` represent annotatio formal pa in declara of the Exe represent object.
`int`	`getParameterCount()`	

		Returns th of formal (whether declared implicitly or neither executabl represent object.
`Class<?>[]`	`getParameterTypes()`	Returns a `Class` obj represent paramete declaratio the execu represent object.
`TypeVariable<Constructor<T>>[]`	`getTypeParameters()`	Returns a `TypeVari` objects th represent variables by the ge declaratio represent `GenericD` object, in order.
`int`	`hashCode()`	Returns a for this `Construc`
`boolean`	`isSynthetic()`	Returns `t` executabl synthetic returns `fa` otherwise
`boolean`	`isVarArgs()`	Returns `t` executabl declared variable n argument `false` oth
`T`	`newInstance(Object... initargs)`	

		Uses the c represent Construc to create initialize a instance c constructo declaring the specif initializati paramete
String	toGenericString()	Returns a describing Construc including paramete
String	toString()	Returns a describing Construc

Methods inherited from class java.lang.reflect.Executable

getAnnotatedExceptionTypes, getAnnotatedParameterTypes, getAnnotationsByType, getParameters

Methods inherited from class java.lang.reflect.AccessibleObject

getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, setAccessible

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.lang.reflect.AnnotatedElement

getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAnnotationPresent

Method Detail

getDeclaringClass

```
public Class<T> getDeclaringClass()
```

Returns the `Class` object representing the class or interface that declares the executable represented by this object.

Specified by:

`getDeclaringClass` in interface `Member`

Specified by:

`getDeclaringClass` in class `Executable`

Returns:

an object representing the declaring class of the underlying member

getName

```
public String getName()
```

Returns the name of this constructor, as a string. This is the binary name of the constructor's declaring class.

Specified by:

`getName` in interface `Member`

Specified by:

`getName` in class `Executable`

Returns:

the simple name of the underlying member

getModifiers

```
public int getModifiers()
```

Returns the Java language modifiers for the executable represented by this object.

Specified by:

`getModifiers` in interface `Member`

Specified by:

`getModifiers` in class `Executable`

Returns:

the Java language modifiers for the underlying member

See Also:

Modifier

getTypeParameters

```
public TypeVariable<Constructor<T>>[] getTypeParameters()
```

Returns an array of `TypeVariable` objects that represent the type variables declared by the generic declaration represented by this `GenericDeclaration` object, in declaration order. Returns an array of length 0 if the underlying generic declaration declares no type variables.

Specified by:

getTypeParameters in interface GenericDeclaration

Specified by:

getTypeParameters in class Executable

Returns:

an array of TypeVariable objects that represent the type variables declared by this generic declaration

Throws:

GenericSignatureFormatError - if the generic signature of this generic declaration does not conform to the format specified in *The Java™ Virtual Machine Specification*

Since:

1.5

getParameterTypes

```
public Class<?>[] getParameterTypes()
```

Returns an array of `Class` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters.

Specified by:

getParameterTypes in class Executable

Returns:

the parameter types for the executable this object represents

getParameterCount

```
public int getParameterCount()
```

Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.

Overrides:

`getParameterCount` in class `Executable`

Returns:

The number of formal parameters for the executable this object represents

Since:

1.8

getGenericParameterTypes

```
public Type[] getGenericParameterTypes()
```

Returns an array of Type objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters.

If a formal parameter type is a parameterized type, the Type object returned for it must accurately reflect the actual type parameters used in the source code.

If a formal parameter type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Overrides:

`getGenericParameterTypes` in class `Executable`

Returns:

an array of Types that represent the formal parameter types of the underlying executable, in declaration order

Throws:

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

`TypeNotPresentException` - if any of the parameter types of the underlying executable refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if any of the underlying executable's parameter types refer to a parameterized type that cannot be instantiated for any reason

Since:

1.5

getExceptionTypes

```
public Class<?>[] getExceptionTypes()
```

Returns an array of `Class` objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object. Returns an array of length 0 if the executable declares no exceptions in its `throws` clause.

Specified by:

`getExceptionTypes` in class `Executable`

Returns:

the exception types declared as being thrown by the executable this object represents

getGenericExceptionTypes

```
public Type[] getGenericExceptionTypes()
```

Returns an array of `Type` objects that represent the exceptions declared to be thrown by this executable object. Returns an array of length 0 if the underlying executable declares no exceptions in its `throws` clause.

If an exception type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Overrides:

`getGenericExceptionTypes` in class `Executable`

Returns:

an array of Types that represent the exception types thrown by the underlying executable

Throws:

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

`TypeNotPresentException` - if the underlying executable's throws clause refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if the underlying executable's throws clause refers to a parameterized type that cannot be instantiated for any reason

Since:

1.5

equals

`public boolean equals(Object obj)`

Compares this `Constructor` against the specified object. Returns true if the objects are the same. Two `Constructor` objects are the same if they were declared by the same class and have the same formal parameter types.

Overrides:

`equals` in class `Object`

Parameters:

`obj - the reference object with which to compare.`

Returns:

`true if this object is the same as the obj argument; false otherwise.`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`public int hashCode()`

Returns a hashcode for this `Constructor`. The hashcode is the same as the hashcode for the underlying constructor's declaring class name.

Overrides:

`hashCode` in class `Object`

Returns:

`a hash code value for this object.`

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

`public String toString()`

Returns a string describing this `Constructor`. The string is formatted as the constructor access modifiers, if any, followed by the fully-qualified name of the declaring class, followed by a parenthesized, comma-separated list of the constructor's formal parameter types. For example:

```
public java.util.Hashtable(int,float)
```

The only possible modifiers for constructors are the access modifiers `public`, `protected` or `private`. Only one of these may appear, or none if the constructor has default (package) access.

Overrides:

`toString` in class `Object`

Returns:

a string describing this Constructor

See *The Java™ Language Specification*:

8.8.3. Constructor Modifiers

toGenericString

```
public String toGenericString()
```

Returns a string describing this `Constructor`, including type parameters. The string is formatted as the constructor access modifiers, if any, followed by an angle-bracketed comma separated list of the constructor's type parameters, if any, followed by the fully-qualified name of the declaring class, followed by a parenthesized, comma-separated list of the constructor's generic formal parameter types. If this constructor was declared to take a variable number of arguments, instead of denoting the last parameter as "*Type*`[]`", it is denoted as "*Type*`...`". A space is used to separate access modifiers from one another and from the type parameters or return type. If there are no type parameters, the type parameter list is elided; if the type parameter list is present, a space separates the list from the class name. If the constructor is declared to throw exceptions, the parameter list is followed by a space, followed by the word "`throws`" followed by a comma-separated list of the thrown exception types.

The only possible modifiers for constructors are the access modifiers `public`, `protected` or `private`. Only one of these may appear, or none if the constructor has default (package) access.

Specified by:

`toGenericString` in class `Executable`

Returns:

a string describing this Constructor, include type parameters

Since:

1.5

See *The Java™ Language Specification*:

8.8.3. Constructor Modifiers

newInstance

```
public T newInstance(Object... initargs)
              throws InstantiationException,
                     IllegalAccessException,
                     IllegalArgumentException,
                     InvocationTargetException
```

Uses the constructor represented by this `Constructor` object to create and initialize a new instance of the constructor's declaring class, with the specified initialization parameters. Individual parameters are automatically unwrapped to match primitive formal parameters, and both primitive and reference parameters are subject to method invocation conversions as necessary.

If the number of formal parameters required by the underlying constructor is 0, the supplied `initargs` array may be of length 0 or null.

If the constructor's declaring class is an inner class in a non-static context, the first argument to the constructor needs to be the enclosing instance; see section 15.9.3 of *The Java™ Language Specification*.

If the required access and argument checks succeed and the instantiation will proceed, the constructor's declaring class is initialized if it has not already been initialized.

If the constructor completes normally, returns the newly created and initialized instance.

Parameters:

`initargs - array of objects to be passed as arguments to the constructor call; values of primitive types are wrapped in a wrapper object of the appropriate type (e.g. a float in a Float)`

Returns:

`a new object created by calling the constructor this object represents`

Throws:

`IllegalAccessException - if this Constructor object is enforcing Java language access control and the underlying constructor is inaccessible.`

`IllegalArgumentException - if the number of actual and formal parameters differ; if an unwrapping conversion for primitive arguments fails; or if, after possible unwrapping, a parameter value cannot be converted to the corresponding formal parameter type by a method invocation conversion; if this constructor pertains to an enum type.`

InstantiationException - if the class that declares the underlying constructor represents an abstract class.

InvocationTargetException - if the underlying constructor throws an exception.

ExceptionInInitializerError - if the initialization provoked by this method fails.

isVarArgs

```
public boolean isVarArgs()
```

Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise.

Overrides:

isVarArgs in class Executable

Returns:

true if an only if this executable was declared to take a variable number of arguments.

Since:

1.5

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this executable is a synthetic construct; returns `false` otherwise.

Specified by:

isSynthetic in interface Member

Overrides:

isSynthetic in class Executable

Returns:

true if and only if this executable is a synthetic construct as defined by *The Java™ Language Specification*.

Since:

1.5

See *The Java™ Language Specification*:

13.1 The Form of a Binary

getAnnotation

```
public <T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Overrides:

`getAnnotation` in class `Executable`

Type Parameters:

`T` - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.5

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotations` in interface `AnnotatedElement`

Overrides:

`getDeclaredAnnotations` in class `Executable`

Returns:

annotations directly present on this element

Since:

1.5

getParameterAnnotations

```
public Annotation[][] getParameterAnnotations()
```

Returns an array of arrays of `Annotation`s that represent the annotations on the formal parameters, in declaration order, of the `Executable` represented by this object. Synthetic and mandated parameters (see explanation below), such as the outer "this" parameter to an inner class constructor will be represented in the returned array. If the executable has no parameters (meaning no formal, no synthetic, and no mandated parameters), a zero-length array will be returned. If the `Executable` has one or more parameters, a nested array of length zero is returned for each parameter with no annotations. The annotation objects contained in the returned arrays are serializable. The caller of this method is free to modify the returned arrays; it will have no effect on the arrays returned to other callers. A compiler may add extra parameters that are implicitly declared in source ("mandated"), as well as parameters that are neither implicitly nor explicitly declared in source ("synthetic") to the parameter list for a method. See `Parameter` for more information.

Specified by:

`getParameterAnnotations` in class `Executable`

Returns:

an array of arrays that represent the annotations on the formal and implicit parameters, in declaration order, of the executable represented by this object

Since:

1.5

See Also:

Parameter, Parameter.getAnnotations()

getAnnotatedReturnType

```
public AnnotatedType getAnnotatedReturnType()
```

Returns an AnnotatedType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable. If this Executable object represents a constructor, the AnnotatedType object represents the type of the constructed object. If this Executable object represents a method, the AnnotatedType object represents the use of a type to specify the return type of the method.

Specified by:

getAnnotatedReturnType in class Executable

Returns:

an object representing the return type of the method or constructor represented by this Executable

Since:

1.8

getAnnotatedReceiverType

```
public AnnotatedType getAnnotatedReceiverType()
```

Returns an AnnotatedType object that represents the use of a type to specify the receiver type of the method/constructor represented by this Executable object. The receiver type of a method/constructor is available only if the method/constructor has a *receiver parameter* (JLS 8.4.1). If this Executable object represents a constructor or instance method that does not have a receiver parameter, or has a receiver parameter with no annotations on its type, then the return value is an AnnotatedType object representing an element with no annotations. If this Executable object represents a static method, then the return value is null.

Overrides:

getAnnotatedReceiverType in class Executable

Returns:

an object representing the receiver type of the method or constructor represented by this Executable

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.